Autonomous DevOps

Project Document

Complete architecture, API reference, and technical documentation
for the LangGraph-powered Self-Healing CI/CD Agent

Version: 1.0.0

Date: July 28, 2026

License: MIT

1. Project Overview

Autonomous DevOps is a self-healing CI/CD agent that automatically detects test failures from GitHub webhooks, analyzes the root cause using LLMs, generates verified fixes, and submits pull requests -- all without human intervention.

The system is built on three core engineering principles: Graph Engineering (LangGraph for stateful agent workflows), Loop Engineering (resilient retry loops with confidence-based early stopping), and Zero-Trust Security (users bring their own keys, no data persistence).

2. System Architecture

The system follows a microservices architecture: FastAPI backend (orchestrator), React frontend (dashboard), and Docker sandbox (execution). Communication via REST + WebSockets.

2.1 Technology Stack

Layer	Technology	Purpose
Orchestration	LangGraph	State machine with typed state & edges
Backend API	FastAPI	Async webhook listener & REST endpoints
LLM Integration	OpenAI / OpenRouter	Code analysis & fix generation
Sandbox	Docker SDK	Isolated test execution per job
Frontend	React 18 + Vite	Real-time dashboard
Styling	TailwindCSS	Responsive dark/light mode
Charts	Recharts	Token/cost metrics visualization
Observability	Langfuse	Tracing & cost monitoring
Deployment	Docker Compose / K8s	Production orchestration

3. LangGraph State Machine

3.1 AgentState (TypedDict)

```
class AgentState(TypedDict):
    repo_url: str           # GitHub repo URL
    commit_sha: str         # Commit SHA
    failure_log: str        # Raw failure log
    stack_trace: str        # Extracted trace
    file_path: str          # Error file
    line_number: int        # Error line
    error_type: str         # Error type
    proposed_fix: str       # AI-generated patch
    code_diff: str          # Unified diff
    test_results: dict      # Sandbox outcome
    pr_url: Optional[str]   # PR link
    confidence_score: float # 0.0-1.0
    attempts: int           # Fix attempt count
    max_attempts: int       # Retry limit
    history: list[str]      # Loop context
    error: Optional[str]    # Error message
    job_id: str             # Unique ID
    agent_thoughts: list[str] # Live stream
```

3.2 Graph Nodes

Node	Description
analyze_failure	Parse logs; regex first, LLM fallback
retrieve_context	Clone repo, read +/-10 lines
generate_fix	LLM generates unified diff patch
sandbox_verify	Run tests in isolated Docker container
submit_pr	Create GitHub PR via API
flag_for_human	Escalate when max retries exceeded

3.3 Conditional Routing

After sandbox_verify:

- Tests Passed -> submit_pr (end)
- Failed & attempts < max -> generate_fix (loop)
- Failed & attempts >= max -> flag_for_human (end)

4. Loop Engineering

4.1 Exponential Backoff

```
async def retry_with_backoff(op, max=3, delay=1.0):
    for attempt in range(1, max+1):
        try: return await op()
        except: await asyncio.sleep(delay)
        delay = min(delay * 2, 30.0)
```

4.2 Early Stopping

The fix loop stops early when:

* Confidence >= 0.7 (fix is good enough)
* Confidence drops > 0.15 in last step (getting worse)
* Max attempts (default 3) reached

4.3 Context Pruning

Token minimisation:

* Only +/-10 lines sent per prompt
* History trimmed to last 5 entries
* Cheap model for analysis (gpt-4o-mini)
* Capable model only for fixes (gpt-4o)

5. REST API Reference

Method	Endpoint	Description
GET	/api/status	Health check
POST	/api/trigger	Manual agent trigger
GET	/api/jobs/{id}	Job details & logs
GET	/api/metrics	Aggregated metrics
POST	/api/config	Update agent settings
POST	/api/session/keys	Store API keys (BYOK)
POST	/webhook/github	GitHub webhook receiver
WS	/ws/jobs/{id}	Real-time log streaming

5.1 Trigger Request

```
POST /api/trigger
{"repo_url":"https://github.com/owner/repo",
 "failure_log":"Error: AssertionError...",
 "max_attempts":3,"model":"gpt-4o"}
```

5.2 WebSocket Protocol

```
// Client connects to /ws/jobs/{job_id}
// Server pushes: {
  "type":"log","job_id":"abc",
  "node":"analyze_failure",
  "message":"Analyzing...",
  "level":"info","timestamp":"..."
}
```

6. Security & Privacy

6.1 BYOK

API keys are stored ephemerally in browser session memory. Keys are sent via Authorization header, never persisted to server disk.

6.2 Sandbox Isolation

Each job uses a dedicated Docker container:

* Network disabled
* CPU limited to 50%
* Memory capped at 512MB
* Read-only source volume
* Automatic cleanup after execution

6.3 Data Retention

Zero data retention: Source code cloned to temp directory, deleted post-job. Job metadata kept in-memory for session duration only.

7. Deployment

7.1 Docker Compose

```
docker compose up --build -d
```

7.2 Kubernetes

```
kubectl create secret generic autodevops-secrets \\
  --from-literal=LLM_API_KEY=sk-... \\
  --from-literal=GITHUB_TOKEN=ghp_...
kubectl apply -f k8s/
```

7.3 Production Checklist

- [ ] Set CORS_ORIGINS to your frontend domain
- [ ] Use proper secrets management (Vault / AWS Secrets Manager)
- [ ] Set up database-backed job persistence
- [ ] Enable HTTPS with proper TLS certificate
- [ ] Configure Prometheus + Grafana monitoring

8. Performance Benchmarks

Metric	Expected	Notes
Time to analyze	2-5s	Regex first, then LLM
Fix generation	5-15s	Depends on model
Sandbox verify	30-120s	First run builds Docker image
Token usage/job	2K-8K	With context pruning
Success rate	70-90%	Varies by complexity

9. License

This project is licensed under the MIT License.